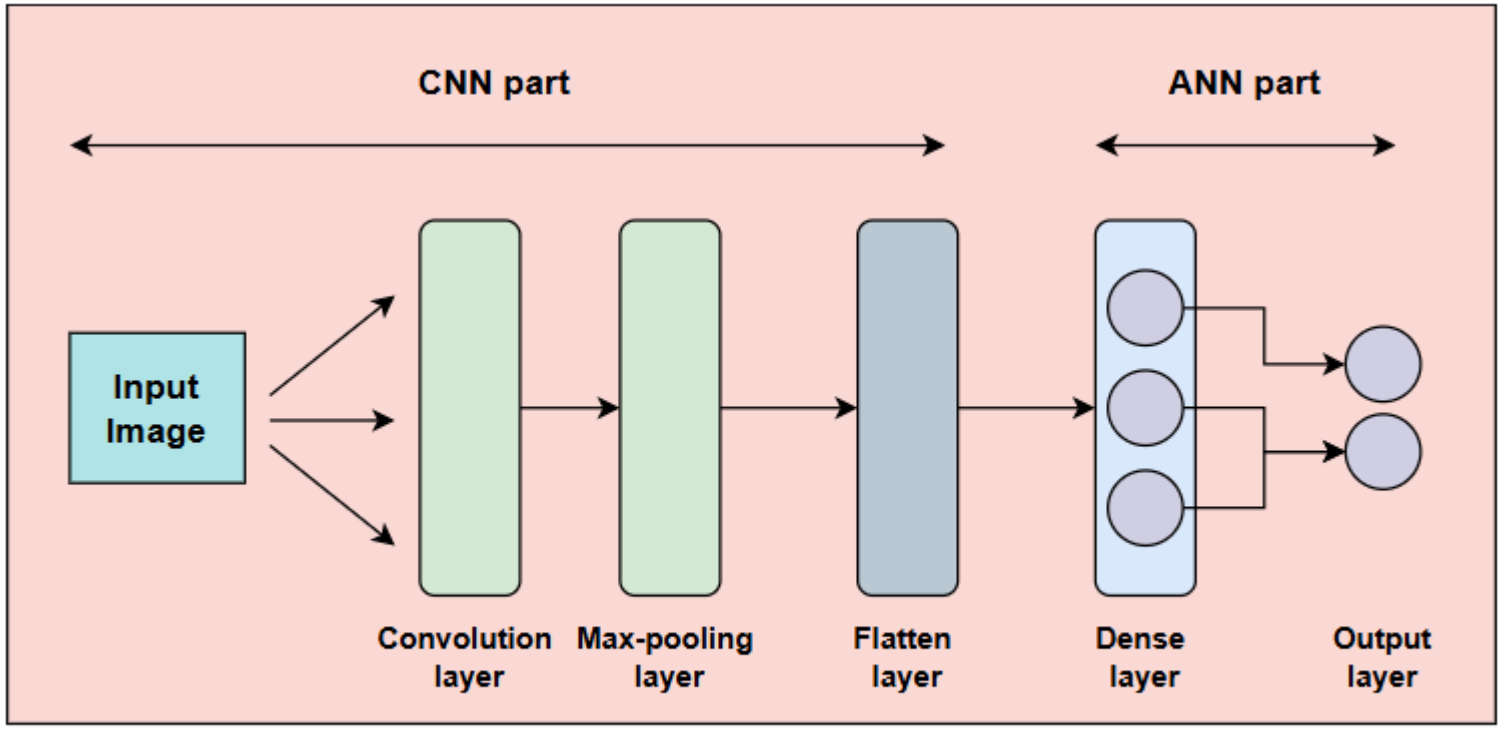
Convolution Neural Network

Part 2

A complete convolutional neural network can be broken down into two parts:

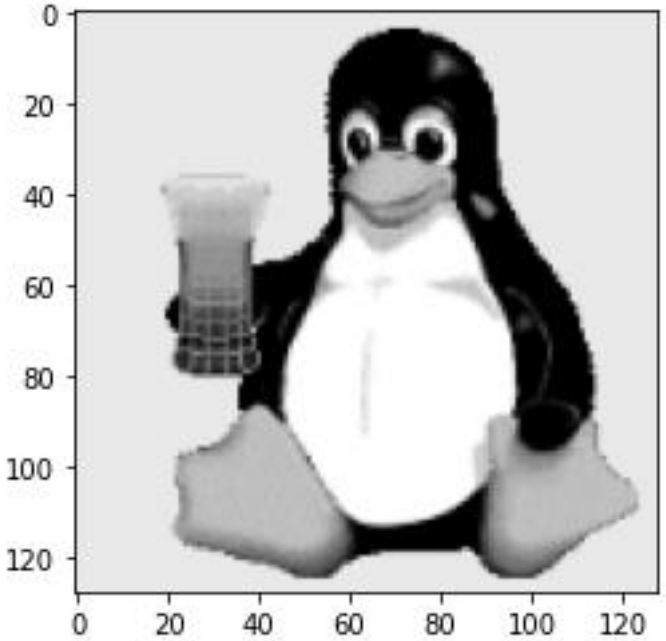
- CNN: The convolutional neural network that comprises the convolutional layers.
- ANN: The artificial neural network that comprises dense layers.

The flatten layer lies between the CNN and the ANN, and its job is to convert the output of the CNN into an input that the ANN can process, as we can see in the diagram below.



Structure of a complete convolutional neural network

Edge detection



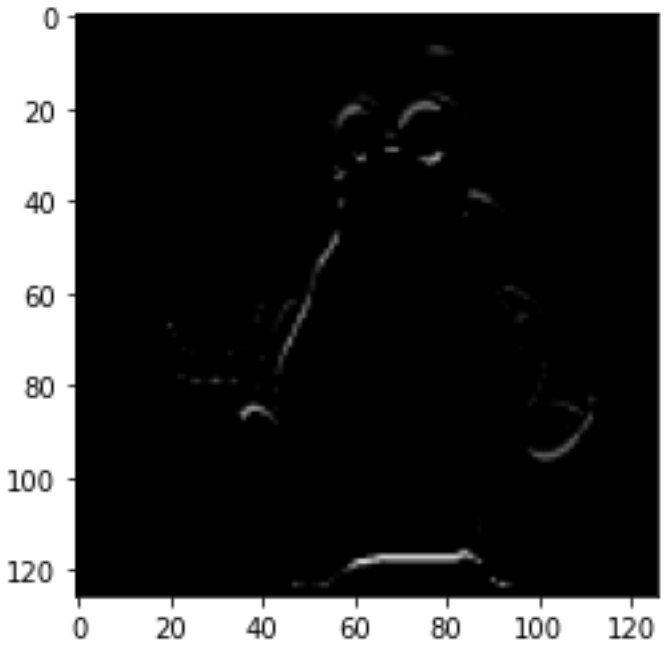
Input image

*

-1	-1	-1
0	0	0
1	1	1

Filter or Kernel

=



Convolution result

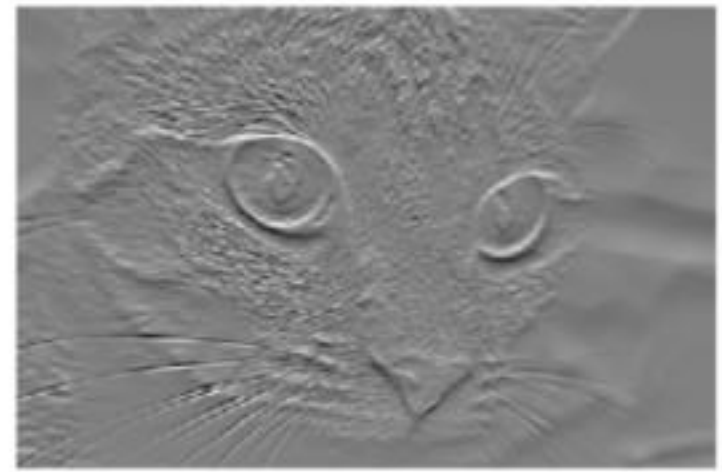
Input image

*

-1	-1	-1
0	0	0
1	1	1

Filter or Kernel

=



Convolution result

Factors that influence how many kernels to use :

Dataset size

•Small dataset (a few thousand images)→ Use *fewer* filters (e.g. 16–32–64) to reduce overfitting.

•Large dataset (tens/hundreds of thousands)→ Can afford *more* filters (32–64–128–256 or more).

Image resolution & complexity

•Small / simple images (e.g. 32×32, MNIST Dataset /CIFAR-like Dataset) → 16–64 filters per layer may be enough.

•Larger or more complex images (128×128, 224×224, real-world scenes Dataset)→ 64–512+ filters per layer are common.

Practical “recipes” used in popular CNNs

You can copy patterns from successful architectures:

- **VGG-like** (for 224×224 images):
  - Block 1: 64 filters
  - Block 2: 128 filters
  - Block 3: 256 filters
  - Block 4: 512 filters
  - Block 5: 512 filters

- **ResNet-like** (for 224×224 images):
  - First conv: 64 filters
  - Then stages with something like 64 → 128 → 256 → 512 (bottlenecks inside).

Simple practical rules you can follow: If you want something actionable:

1.For small images (≤ 64×64), small dataset

1. Conv layer (block 1): 16–32 filters
2. Conv layer (block 2): 32–64 filters
3. Conv layer (block 3): 64–128 filters

2.For medium images (≈ 128×128), medium dataset

1. Conv layer (block 1): 32–64 filters
2. Conv layer (block 2): 64–128 filters
3. Conv layer (block 3): 128–256 filters

3.For larger images (≈ 224×224) or complex tasks

1. Conv layer (block 1): 64 filters
2. Conv layer (block 2): 128 filters
3. Conv layer (block 3): 256 filters
4. Conv layer (block 4): 512 filters

Types of Kernels

Different types of kernels are used to extract different features from the input data. Some common types of kernels include:

Edge Detection Kernels

These kernels are designed to highlight vertical, horizontal or diagonal edges within an image which are important for recognizing objects. Some of them are:

Sobel Filter: Used to find horizontal or vertical edges. It focuses pixels where there are sudden intensity changes.

a vertical Sobel Kernel:

[-1, 0, 1]
[-2, 0, 2]
[-1, 0, 1]

a horizontal Sobel Kernel:

[1, 2, 1]
[0, 0, 0]
[-1,-2,-1]

•Prewitt Filter: Similar to Sobel but uses a different weighting in the matrix to detect edges.

a Vertical Prewitt Kernel:

[-1, 0, 1]
[-1, 0, 1]
[-1, 0, 1]

a horizontal Prewitt Kernel:

[1, 1, 1]
[0, 0, 0]
[-1,-1,-1]

•Laplacian Filter: Detects edges based on the second derivative of the image which provides a more detailed detection that includes diagonal edges.

[0, -1, 0]
[-1, 5,-1]
[0, -1, 0]

Building and Using a Convolutional Neural Network (CNN) for Image Classification with Keras and TensorFlow

Step 1: Importing Libraries and Packages

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import image_dataset_from_directory
```

Step 2: Initializing the CNN

```
classifier = Sequential()
```

create a sequential model using Sequential class from Keras.

Step 3: Adding Convolutional Layers

```
classifier.add(Conv2D(32, (3, 3), input_shape=(64, 64, 3), activation='relu'))
classifier.add(MaxPooling2D(pool_size=(2, 2)))

# Add another convolutional layer
classifier.add(Conv2D(32, (3, 3), activation='relu'))
classifier.add(MaxPooling2D(pool_size=(2, 2)))
```

Step 5: Flattening

```
classifier.add(Flatten())
```

Step 6: Adding Fully Connected Layers

```
classifier.add(Dense(units=128, activation='relu'))
classifier.add(Dense(units=256, activation='relu'))
classifier.add(Dense(units=1, activation='sigmoid'))
```

Step 7: Compiling the Model

```
classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Step 8: Training the Model:

```
classifier.fit(train_images, train_labels, epochs=150, …)
```